

GPUMODE Lecture Study Notes: Lecture 41

FlashInfer

Core Problem the Lecture Addresses

This lecture explains **FlashInfer**, an attention and inference-kernel system designed for efficient, customizable large language model inference serving. The core problem is that production autoregressive inference does not look like a single clean dense attention problem. It involves variable-length requests, fragmented key-value caches, prefix sharing, speculative decoding trees, page-table-style memory management, custom attention variants, and strict latency constraints.

The central question is:

How can an inference engine provide high-performance attention kernels while supporting many different key-value cache layouts, sparse attention patterns, runtime sequence length distributions, and model-specific attention variants?

The lecture argues that hard-coding one attention kernel for one cache layout is too restrictive. FlashInfer instead builds around two ideas:

- **Unified storage abstraction:** model key-value cache access as block-sparse attention.
- **Programmable compute abstraction:** generate specialized CUDA or CUTLASS-based kernels from user-specified attention variants.

This is motivated by the observation that modern LLM serving is not just a model execution problem. It is a memory-management, scheduling, sparse-layout, and kernel-customization problem.

Required Background

Autoregressive Inference

In autoregressive inference, a model generates tokens one at a time. Given a prompt of length L , the model first processes the full prompt during the **prefill** stage. Then it generates future tokens during the **decode** stage.

Let Q , K , and V denote query, key, and value tensors. Standard attention is:

$$S = QK^T,$$

$$P = \text{softmax}(S),$$

$$O = PV.$$

During prefill, many queries attend to many keys. During decode, each request often has only one new query token but attends to a potentially long key-value cache.

Key-Value Cache

The **key-value cache**, usually called the KV cache, stores previous K and V tensors so that the model does not recompute them for every newly generated token.

For a transformer with N_{layers} layers, H_{kv} key-value heads, head dimension D , sequence length L , and element size b bytes, the approximate KV cache size is:

$$\text{KV bytes} = 2 \cdot N_{\text{layers}} \cdot H_{\text{kv}} \cdot L \cdot D \cdot b.$$

The factor 2 appears because both keys and values are stored. This memory can dominate serving cost for long contexts or high request concurrency.

Paged Attention

Paged attention, popularized by vLLM, stores the KV cache in fixed-size pages instead of requiring one contiguous allocation per request. A request’s logical sequence is represented by a page table mapping logical token blocks to physical KV-cache blocks.

If each page stores B_{page} tokens, then a sequence of length L requires approximately:

$$N_{\text{pages}} = \left\lceil \frac{L}{B_{\text{page}}} \right\rceil.$$

Paged attention reduces memory fragmentation and allows the serving engine to reuse operating-system-like ideas such as page tables and swapping.

Radix Attention and Prefix Caching

Some serving systems exploit repeated prefixes. For example, many requests may share the same system prompt. Radix attention stores shared prefixes in a radix-tree-like data structure. Instead of storing duplicate KV cache entries for every request, shared prefix blocks can be reused.

The important distinction is:

- Standard paged attention usually uses pages with several tokens, such as 16 tokens.
- Radix-style prefix caching may use a page size of 1 token to reduce fragmentation and improve prefix cache hit rate.

Speculative Decoding

Speculative decoding uses a smaller draft model to propose several candidate tokens. A larger target model verifies those candidates in parallel. Some methods generate not merely a chain of candidates but a **tree** of candidate continuations.

This creates attention masks that are neither purely triangular nor purely dense. A candidate token may attend only to its ancestors in the proposal tree. Therefore, the attention pattern becomes naturally sparse.

Main Concepts

Why FlashInfer Needs a Unified Attention Engine

The lecture identifies several attention and KV-cache design challenges:

1. **Paged KV cache:** requests have variable lengths and are stored in pages.
2. **Radix-tree prefix reuse:** many requests share long prefixes.
3. **Speculative decoding trees:** verification attention follows tree-shaped masks.
4. **KV-cache pruning or compression:** methods such as Quest select only important pages.
5. **Attention variants:** modern models may use grouped-query attention, multi-query attention, sliding-window masks, logit caps, sigmoid attention, fused RoPE, or other custom transformations.

A single dense FlashAttention-style kernel is not enough because it assumes regular memory layout and a relatively fixed attention computation. FlashInfer tries to provide a kernel system that can adapt to these cases without rewriting the entire attention library each time.

Block-Sparse Attention as a Storage Abstraction

FlashInfer models the KV-cache access pattern as a **block-sparse matrix**. In ordinary sparse matrices, the smallest unit is a scalar. In block-sparse matrices, the smallest unit is a dense block.

Let the logical attention matrix be A . A block-sparse representation partitions it into blocks of size $B_r \times B_c$. A block is active if any useful attention computation exists inside it.

The sparse block structure can be represented with a compressed sparse row format for blocks:

`indptr[i]` stores the starting offset of row i ,

`indices[j]` stores the column index of a nonzero block,

`values[j]` optionally stores block-level masks or metadata.

This is analogous to CSR or BSR sparse matrix formats, but adapted to attention and KV-cache layouts.

Paged Attention as Block-Sparse Attention

For a batch of requests, each request corresponds to a row region in a logical sparse attention matrix. The columns represent logical or physical KV-cache pages. A request attends only to the pages assigned to it.

Therefore, paged attention can be seen as a block-sparse attention problem:

request $\longrightarrow$ nonzero page blocks in the KV cache.

This view unifies page-table-based KV-cache management with sparse matrix computation.

Vector-Sparse Blocks

A key design choice in FlashInfer is supporting flexible block shapes, especially small column-block sizes.

A traditional block-sparse tensor-core kernel may use blocks such as 16×16 or 128×128 . This is tensor-core friendly, but it can waste computation if the true attention mask is very sparse. For example, if only one entry in a 16×16 block is useful, activating the whole block performs 256 element computations to keep only one result.

The lecture describes a more flexible approach where the block can have one column:

$$B_r \times B_c = 16 \times 1.$$

Then a single useful entry activates only a vector-like block. The waste changes from approximately:

$$\frac{255}{256}$$

to approximately:

$$\frac{15}{16}.$$

This is still imperfect, but it is much better for sparse tree attention or fine-grained sparse masks.

Why Tensor Cores Can Still Be Used

A common misconception is that tensor cores require globally contiguous dense matrices. The lecture clarifies that this is not strictly true.

The kernel can gather non-contiguous rows or columns from global memory into shared memory. Once the data is in shared memory, it can be arranged densely and fed into tensor-core matrix multiply instructions.

The important property in attention is that the head dimension is contiguous. For a key tensor, the last dimension is usually the head dimension D , and memory accesses along this dimension are contiguous. Therefore, even sparse row selection can still issue coalesced or nearly coalesced loads for each selected row.

Composable Sparse Formats for Shared Prefixes

Prefix sharing creates a different structure. Many requests share the same KV-cache region. Instead of treating everything as small blocks, FlashInfer can decompose the sparse attention pattern into several sparse matrices with different block sizes.

Conceptually:

$$A = A_{\text{shared-prefix}} + A_{\text{unique-suffix}}.$$

The shared-prefix part may use larger blocks because many requests access the same keys. The unique suffix part may use smaller blocks because each request has request-specific tokens.

This decomposition allows the kernel to use software-managed reuse for the shared prefix. The same KV-cache data can be loaded into shared memory and reused across multiple queries or requests.

Hardware-Level Explanation

Memory Hierarchy

FlashInfer is designed around GPU memory hierarchy:

- **HBM:** large global memory, high bandwidth, high latency.
- **L2 cache:** shared across streaming multiprocessors.
- **L1 cache and shared memory:** on-chip memory near each SM.
- **Registers:** fastest storage, private to each thread.
- **Tensor cores:** matrix-multiply units specialized for dense tile operations.

Attention over long KV caches is often memory-bandwidth-bound because the kernel must read large amounts of K and V data while doing relatively little computation per byte in decode.

Global Memory to Shared Memory Copies

For dense KV-cache storage, the global-to-shared-memory mapping is regular. On Hopper GPUs, Tensor Memory Accelerator, or TMA, can move two-dimensional tiles efficiently.

For sparse KV-cache storage, the mapping is indirect:

$$\text{logical block} \longrightarrow \text{index array} \longrightarrow \text{physical memory address}.$$

This indirection means TMA is not always applicable, because the source addresses are not a simple affine two-dimensional tile. FlashInfer therefore often falls back to Ampere-style asynchronous copy instructions, such as `ldgsts`, for sparse layouts.

The trade-off is:

- Dense layout can use TMA and reduce pointer arithmetic.
- Sparse layout needs extra address calculations and may increase register pressure.

Register Pressure

Sparse access requires more registers for pointers, offsets, indices, masks, and intermediate metadata. If register usage is too high, the compiler may spill registers into local memory. Register spilling is expensive because local memory usually lives in global memory.

Therefore, FlashInfer must tune tile sizes and pipeline structure to avoid spills. On FlashAttention 3 style Hopper kernels, sparse access can have a larger gap from dense performance because dense kernels use TMA while sparse kernels require more explicit address computation.

SM Scheduling and Persistent Kernels

FlashInfer uses runtime scheduling to balance work across CTAs. A CTA, or cooperative thread array, is essentially a CUDA thread block.

For some H100 kernels, one CTA may occupy most of one SM because the kernel uses many registers and shared memory. If one CTA maps to one SM, launching approximately:

$$N_{CTA} = N_{SM} \cdot N_{CTA \text{ per SM}}$$

helps keep all SMs busy.

FlashInfer’s scheduler divides attention work into tiles and assigns them to CTAs so that each CTA receives roughly equal total work.

Mathematical Reasoning

Attention Arithmetic Intensity

Arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

A decode attention step with one query and long KV cache has low arithmetic intensity. For each key-value element, the kernel reads memory and performs a small number of operations. Therefore, decode attention is often bandwidth-bound.

In grouped-query attention, multiple query heads share the same KV head. If G query heads share one KV head, the same K and V data can be reused across G query heads. The effective arithmetic intensity increases roughly with G , assuming the implementation reuses the loaded KV data efficiently.

Prefill, Decode, and Append

The lecture distinguishes three regimes.

Prefill

In prefill:

$$L_q = L_k.$$

The attention mask is triangular for causal attention. There are many queries, so the kernel has substantial matrix-multiply work.

Decode

In decode:

$$L_q = 1, \quad L_k \gg 1.$$

This is usually memory-bound because the kernel streams through the KV cache for a single new token.

Append or Incremental Prefill

In append:

$$1 < L_q < L_k.$$

This occurs in speculative decoding or chunked prefill, where a small number of new tokens are appended to an existing context.

Load-Balancing Objective

Suppose a batch contains requests with different attention costs $c_1, c_2, \dots, c_n$. FlashInfer splits large requests into work tiles and assigns tiles to CTAs.

The objective is to minimize the maximum accumulated CTA cost:

$$\min \max_j \sum_{i \in \mathcal{W}_j} c_i,$$

where $\mathcal{W}_j$ is the set of work tiles assigned to CTA j .

The lecture describes a greedy strategy:

1. Split all requests into work tiles.

2. Sort work tiles from largest to smallest.
3. Maintain a priority queue of CTAs by accumulated work.
4. Assign the largest remaining tile to the CTA with the least accumulated work.

This resembles classic list scheduling for makespan minimization.

Split-K Attention and Determinism

For long KV caches, one request may be split across multiple KV chunks. Each chunk produces a partial output. Then a second kernel merges the partial outputs.

If partial reductions are accumulated with atomics, the floating-point accumulation order may vary across runs. Floating-point addition is not associative:

$$(a + b) + c \neq a + (b + c)$$

in finite precision. Therefore, nondeterministic accumulation order can produce different outputs.

FlashInfer avoids atomic aggregation in its default scheduler so that the merge order is deterministic for the same scheduling input.

Code Walkthrough

Representative Paged KV-Cache Lookup

The following pseudocode shows the core logic of indirect KV-cache access.

```
int request_id = blockIdx.x;
int tile_id = blockIdx.y;

int logical_page = tile_id;
int physical_page = page_table[request_id][logical_page];

float* k_ptr = k_cache + physical_page * page_size * head_dim;
float* v_ptr = v_cache + physical_page * page_size * head_dim;

// Threads cooperatively load K and V rows into shared memory.
for (int offset = threadIdx.x; offset < page_size * head_dim; offset += blockDim.x) {
    shared_k[offset] = k_ptr[offset];
    shared_v[offset] = v_ptr[offset];
}

__syncthreads();
```

```
// After this point, tensor-core or CUDA-core computation can consume  
// the shared-memory tile as if it were dense.
```

The important hardware idea is that the global memory source may be sparse or indirect, but the shared-memory destination is dense. This converts irregular global layout into regular on-chip computation.

Representative Block-Sparse Metadata

```
struct BlockSparseLayout {  
    int* indptr;    // Row pointer array.  
    int* indices;   // Nonzero block column indices.  
    int* masks;     // Optional block masks or metadata.  
    int num_rows;  
    int block_rows;  
    int block_cols;  
};
```

The `indptr` array identifies which nonzero blocks belong to each row. The `indices` array identifies where those blocks are located in the logical KV-cache address space. Optional masks allow FlashInfer to express finer-grained attention restrictions inside each block.

Representative Custom Attention Functor

FlashInfer's JIT system allows users to describe attention variants with small pieces of CUDA-like code. A simplified example for sigmoid attention is:

```
struct SigmoidAttention {  
    __device__ float logits_transform(float score) const {  
        return 1.0f / (1.0f + expf(-score));  
    }  
  
    __device__ bool logits_mask(int q_idx, int k_idx) const {  
        return k_idx <= q_idx;  
    }  
};
```

In standard attention, scores are transformed with softmax over a row. In sigmoid attention, the score transformation is different. The lecture emphasizes that many attention variants can be expressed by modifying score transforms, masks, query transforms, or reduction behavior.

Runtime Wrapper API

The FlashInfer serving interface is organized around three stages:

```

wrapper = AttentionWrapper(task_info, jit_args)

wrapper.plan(
    q_indptr=q_indptr,
    kv_indptr=kv_indptr,
    block_sparse_layout=layout
)

output = wrapper.run(
    query=q,
    key_cache=k_cache,
    value_cache=v_cache
)

```

The `init` stage creates a wrapper and optionally compiles a specialized kernel. The `plan` stage computes scheduling metadata for the current generation step. The `run` stage executes the attention kernel using the planned work assignment. The plan can often be reused across transformer layers because many layers share the same KV-cache layout. This amortizes the planning cost.

Performance Intuition

Dense Versus Sparse KV Cache

Sparse KV-cache layouts are more flexible but introduce overhead:

- extra index loads,
- extra pointer arithmetic,
- higher register pressure,
- less opportunity to use TMA,
- more complicated scheduling.

However, sparse layouts are necessary for paged attention, prefix caching, speculative decoding trees, and KV-cache pruning.

The performance goal is not always to beat dense contiguous attention. The goal is to get close to dense performance while enabling layouts that dense kernels cannot support efficiently.

Why Variable-Length Batches Hurt

If a batch contains both short and long requests, assigning one request per CTA can lead to poor load balance. Some CTAs finish quickly while others process long sequences. This creates **wave quantization**, where some SMs become idle while others continue working.

Runtime scheduling improves utilization by splitting long requests and distributing work more evenly.

Why Prefix Sharing Helps

If N requests share a prefix of length L_{prefix} , naive execution may load the same prefix KV data N times. With software-managed reuse, the kernel can load shared prefix blocks once and reuse them across multiple requests.

The potential bandwidth reduction is roughly:

$$\text{reuse factor} \approx N$$

for the shared prefix portion, though the actual speedup depends on attention dominance, kernel overhead, and serving-framework overhead.

Why Fusing RoPE Can Help

Some models cannot pre-apply rotary position embeddings to keys before storing them in the KV cache. In such cases, RoPE must be applied during attention. If RoPE is implemented as a separate kernel, it adds memory traffic and launch overhead.

A fused RoPE attention kernel performs:

$$K_{\text{rotated}} = \text{RoPE}(K)$$

inside the attention kernel, avoiding extra reads and writes.

Common Misconceptions

Misconception: Paged Attention Is the Final KV-Cache Design

Paged attention is powerful, but it is only one point in the design space. Prefix caching, tree-structured speculative decoding, cache pruning, cache compression, and multi-tier GPU-CPU-SSD cache systems are still active research areas.

Misconception: Tensor Cores Require Fully Dense Global Memory

Tensor cores require dense tile operands at the point of matrix multiply. They do not require that the original global memory layout be dense. A kernel can gather sparse global rows into shared memory and then use tensor cores on the dense shared-memory tile.

Misconception: Custom Attention Always Requires Rewriting the Whole Kernel

FlashInfer tries to separate the attention template from user-defined transformations. Many variants can be represented by custom score transforms, masks, query transforms, and epilogues.

Misconception: Maximum Throughput Always Means Reproducibility

High-performance reductions may change accumulation order depending on batching, splitting, and scheduling. Fully reproducible serving may require disabling some split-K or atomic-style reductions, sacrificing performance for deterministic outputs.

Practical Takeaways

- Treat KV-cache layout as a first-class systems problem.
- Use paged or block-sparse abstractions when requests have variable length.
- Use prefix-aware layouts when many requests share prompts.
- Use vector-sparse or small-column block formats for tree-shaped speculative decoding masks.
- Separate compile-time specialization from runtime scheduling.
- Tune tile sizes separately for prefill, decode, and append workloads.
- Watch register pressure carefully when adding sparse metadata or pointer arithmetic.
- Avoid atomic floating-point aggregation when deterministic outputs are required.
- Reuse planning metadata across layers whenever the KV-cache layout is shared.
- Benchmark under realistic sequence length distributions, not only constant-length batches.

Project or Experiment Ideas

Experiment 1: Dense Versus Paged KV Cache

Implement a small attention benchmark where one version reads contiguous KV cache and another reads through a page table. Measure bandwidth and latency as sequence length varies.

Experiment 2: Variable-Length Load Balancing

Generate synthetic batches with constant, uniform, and Zipf-like sequence length distributions. Compare one-request-per-block scheduling against greedy tile scheduling.

Experiment 3: Prefix Sharing

Create a batch where N requests share a long prefix but have short unique suffixes. Benchmark naive repeated prefix loading against a shared-prefix-aware approach.

Experiment 4: Reproducibility

Run attention with and without split-K reductions. Compare output differences when the same request is executed alone versus inside a larger batch.

Experiment 5: Custom Attention Variant

Implement a toy sigmoid attention or sliding-window attention variant in PyTorch, Triton, and CUDA. Compare how much code must change in each implementation.

Final Mental Model

FlashInfer is best understood as a bridge between irregular inference-serving workloads and high-performance GPU kernels. Modern LLM serving creates messy attention problems: paged KV caches, shared prefixes, speculative decoding trees, pruned cache pages, custom masks, and variable request lengths. FlashInfer turns these cases into a unified block-sparse representation and then generates specialized attention kernels that still try to use GPU hardware efficiently.

The mental model is: represent irregular KV-cache access as sparse structure, gather irregular global-memory data into dense on-chip tiles, compute with tensor cores or CUDA cores, and use runtime scheduling so all SMs finish their assigned attention work at roughly the same time.

Visual Concept

Draw a serving engine feeding a batch of variable-length requests into a FlashInfer wrapper. Show the wrapper splitting into a block-sparse KV-cache layout, a planning stage that assigns work tiles to CTAs, and a run stage where SMs load sparse global KV blocks into shared memory before executing dense tensor-core attention.